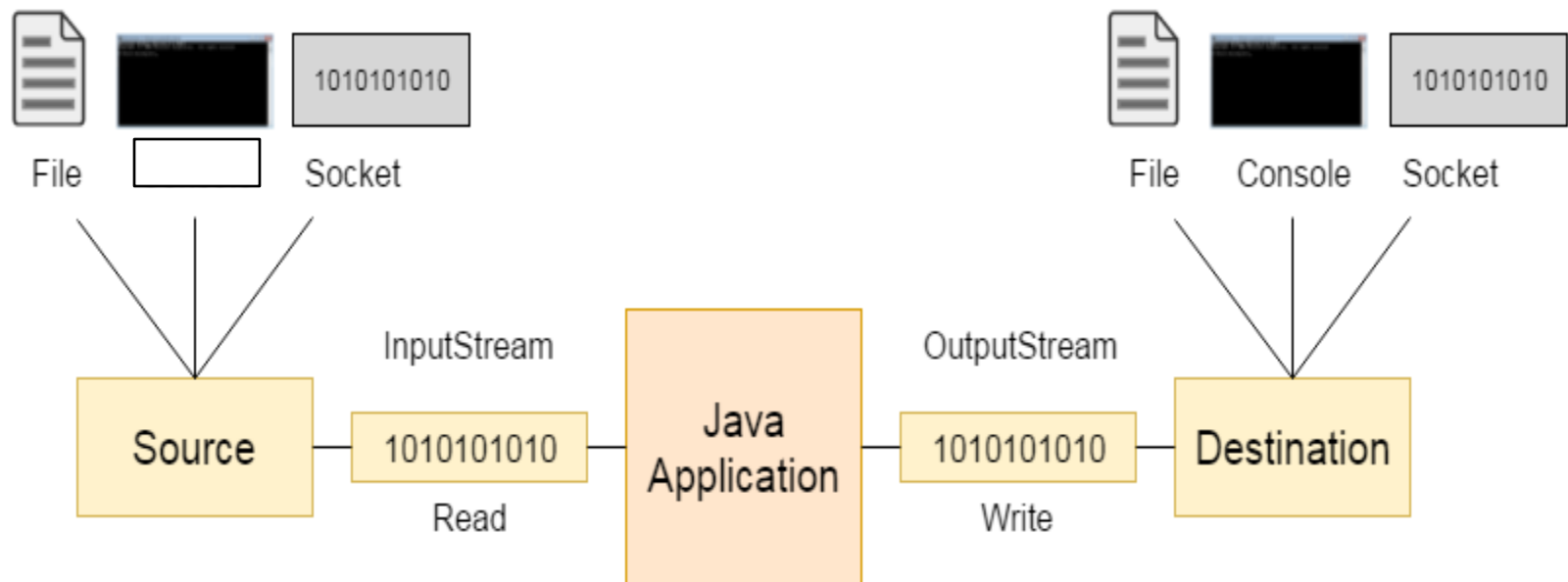


IO Streams



PresentationPoint



- There are many classes and interfaces in the **java.io** package.

File

- A **File** object is used to obtain or manipulate the information associated with a disk file.
- Ex: permissions, time, date, and directory path etc.
- **File** class **does not** specify how information is retrieved from or stored in files;
- **It describes the properties of a file itself.**

File

- A directory in Java is treated simply as a **File** with one additional property:
- List of filenames that can be examined by the **list()** method.

Constructors used to create **File** objects:

`File(String directoryPath)`

`File(String directoryPath, String filename)`

`File(File dirObj, String filename)`

`File(URI uriObj)`

Constructors used to create **File** objects:

```
File f1 = new File("c:/mit/ict/cce/prg1.txt");
```

```
File f2 = new File(" c:/mit/ict/cce/", "prg1.txt");
```

```
File f3 = new File(" c:/mit/ict/cce/");
```

```
File f4 = new File(f3,"prg1.txt");
```

Windows:

recognize forward slash(/).

If we use backaword slash, we need to use escape sequence(\\)

Unix :

recognize backword slash(\).

Methods :

String getName()

String getParent()

String getPath()

String getAbsolutePath()

Methods :

boolean exists()

boolean canWrite()

boolean canRead()

boolean isDirectory()

boolean isFile()

boolean isAbsolute()

Methods :

long lastModified()

long length()

Methods :

`boolean renameTo(File newName)`

- *newName* becomes the new name of the invoking **File** object.
- It will return **true** upon success and **false** if the file cannot be renamed.
(if you either attempt to rename a file so that it uses an existing filename).

Creating Directories

`boolean mkdir( )`

`boolean mkdirs( )`

`mkdirs` : To create a directory for which no path exists. It creates a directory and all the parents of the directory.

Creating Directories

```
File f = new File("D:\\MIT");
```

```
if (f.mkdir())
```

```
{  
    System.out.println("Directory is created");  
}
```

```
else
```

```
{  
    System.out.println("Directory cannot be created");  
}
```

Boolean delete()

To delete file and directory.

To delete directory, it should be empty

Directories

```
String [] list()
```


The `listFiles( )` Alternative

- Variation to the **`list( )`** method.
- Which returns `File` instead of string

```
File[ ] listFiles( )
```

```
File[ ] listFiles(FilenameFilter FFObj)
```

Using FilenameFilter

- To limit the number of files returned by the **list()** method.
- To include only those files that match a certain filename pattern, or *filter*.
- Second form of **list()** :

String[] **list**(FilenameFilter *FFObj*)

- FFObj is an object of a class that implements the **FilenameFilter** interface.

- **FilenameFilter** defines a single method, **accept()**, which is called once for each file in a list.

boolean accept(File *directory*, String *filename*)

- The **accept()** method returns **true** for files in the directory specified by *directory* that should be included in the list (that is, those that match the *filename* argument), and returns **false** for those files that should be excluded.
- The **OnlyExt** class implements **FilenameFilter**.

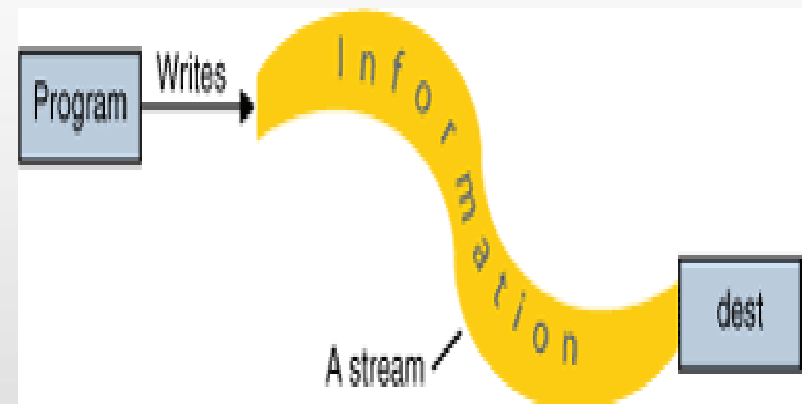
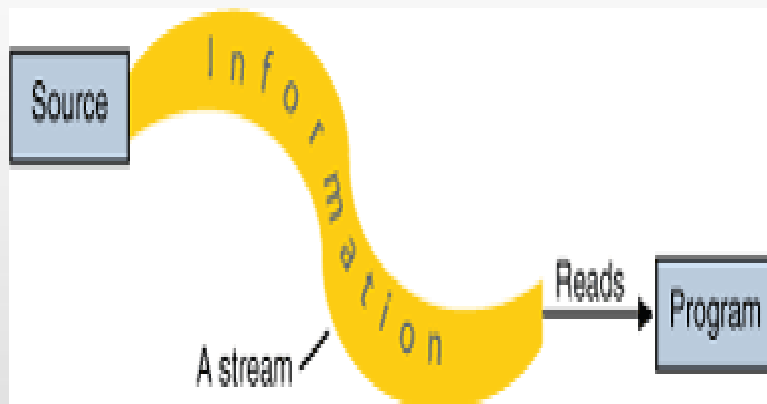
```
import java.io.*;

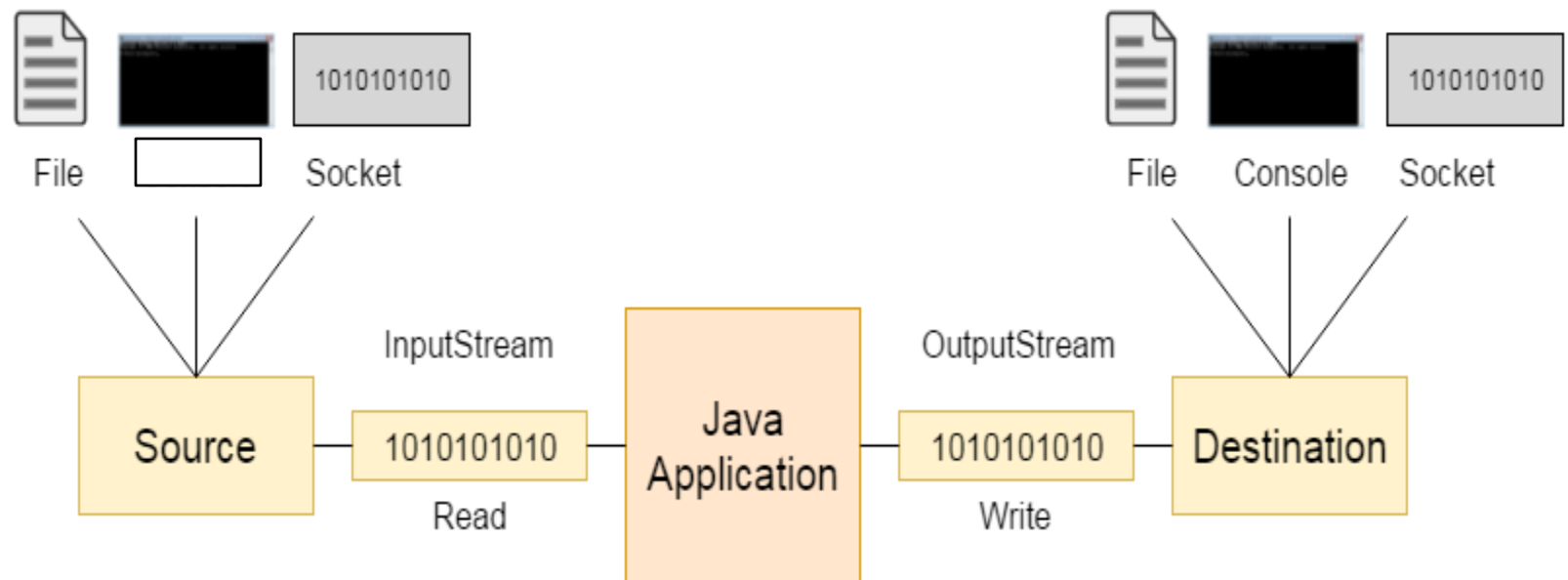
public class OnlyExt implements FilenameFilter
{
    String ext;
    public OnlyExt(String ext)
    {
        this.ext = "." + ext;
    }
    public boolean accept(File dir, String name)
    {
        return name.endsWith(ext);
    }
}
```

```
import java.io.*;

class DirListOnly
{
    public static void main(String args[])
    {
        String dirname = "/java";
        File f1 = new File(dirname);
        FilenameFilter only = new OnlyExt("html");
        String s[] = f1.list(only);
        for (int i=0; i < s.length; i++)
        {
            System.out.println(s[i]);
        }
    }
}
```

- A stream is a logical entity that either produces or consumes information.
- A stream is linked to a physical device by the Java I/O system.





File: "Hello"

Storage in File (Bytes):

```
+---+---+---+---+---+---+
| 72 |101 |108 |108 |111 | 10 |
+---+---+---+---+---+---+
H   e   l   l   o   \n
```

```
|
| FileInputStream (reads one byte at a time)
v
```

Program Receives:

72 → 101 → 108 → 108 → 111 → 10

File: "Hello"

Storage in File (Bytes):

```
+---+---+---+---+---+---+
| 72 |101|108|108|111| 10 |
+---+---+---+---+---+---+
```

```
|
| FileReader (decodes bytes to characters)
v
```

Program Receives:

'H' → 'e' → 'l' → 'l' → 'o' → '\n'

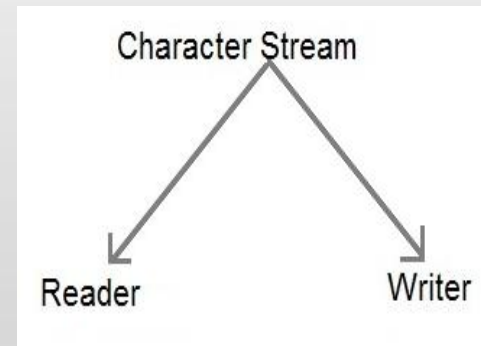
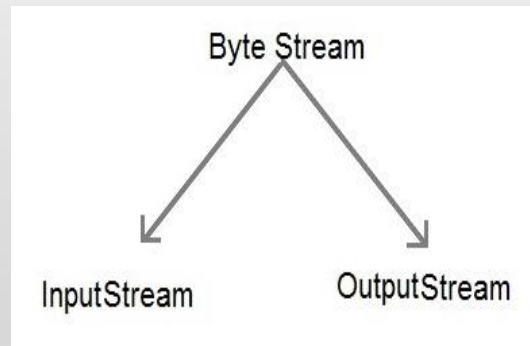
The Stream Classes

- Java's stream-based I/O is built upon four **abstract** classes:

InputStream,
OutputStream,
Reader,
Writer.

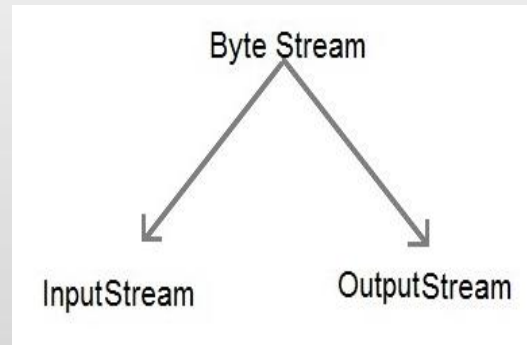
- Used to create several concrete stream subclasses.
- Programs perform their I/O operations through subclasses.
- The top-level classes define the basic functionality common to all stream classes.

- **InputStream** and **OutputStream** are designed for byte streams.
- **Reader** and **Writer** are designed for character streams.
- use the character stream classes when working with characters or strings, and use the byte stream classes when working with bytes or other binary objects.



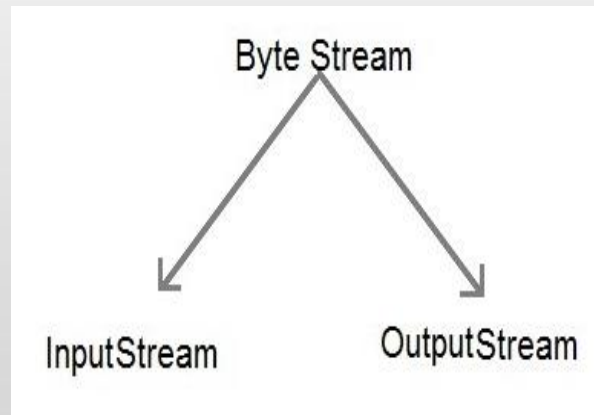
The Byte Streams

- Byte stream classes provide a rich environment for handling byte-oriented I/O.



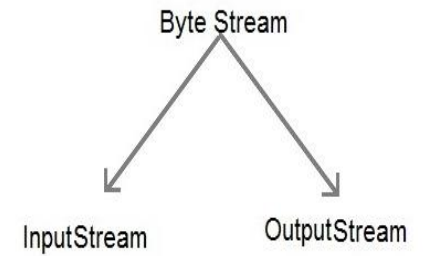
InputStream

- **InputStream** is an **abstract class** that defines Java's model of streaming byte input.
- Most of the methods in this class will throw an **IOException** on error conditions.



methods in **InputStream**.

`int read( )`

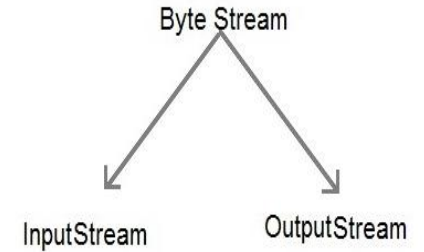


- Returns an integer representation of the next available byte of input.

methods in **InputStream**.

`int read( )`

`int read(byte buffer[ ])`



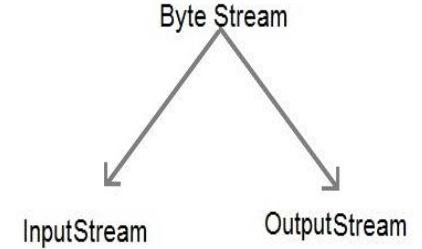
- Attempts to read up to `buffer.length` bytes into buffer and **returns the actual number of bytes that were successfully read.**

methods in **InputStream**.

`int read( )`

`int read(byte buffer[ ])`

`int read(byte buffer[ ], int offset, int numBytes)`



- Attempts to read up to numBytes bytes into buffer starting at buffer[offset], returning the number of bytes successfully read.
- **-1 is returned** when the end of the file is encountered.

methods in **InputStream**.

`int available( )`

- Returns the number of bytes of input currently available for reading.

methods in **InputStream**.

int available()

long skip(long numBytes)

- Ignores (that is, skips) numBytes bytes of input, returning the number of bytes actually ignored.

methods in **InputStream**.

`int available( )`

`long skip(long numBytes)`

`void close( )`

- Closes the input source. Further read attempts will generate an `IOException`.

methods in **InputStream**.

boolean markSupported()

void mark(int numBytes)

void reset()

- Returns true if mark()/reset() are supported by the invoking stream.
- Places a mark at the **current point** in the input stream that will remain valid until numBytes bytes are read.
- Resets the input pointer to the previously set mark.

Table 19-1 shows the methods in **InputStream**.

Method	Description
int available()	Returns the number of bytes of input currently available for reading.
void close()	Closes the input source. Further read attempts will generate an IOException .
void mark(int <i>numBytes</i>)	Places a mark at the current point in the input stream that will remain valid until <i>numBytes</i> bytes are read.
boolean markSupported()	Returns true if mark()/reset() are supported by the invoking stream.
int read()	Returns an integer representation of the next available byte of input. -1 is returned when the end of the file is encountered.
int read(byte <i>buffer</i> [])	Attempts to read up to <i>buffer.length</i> bytes into <i>buffer</i> and returns the actual number of bytes that were successfully read. -1 is returned when the end of the file is encountered.
int read(byte <i>buffer</i> [], int <i>offset</i> , int <i>numBytes</i>)	Attempts to read up to <i>numBytes</i> bytes into <i>buffer</i> starting at <i>buffer[offset]</i> , returning the number of bytes successfully read. -1 is returned when the end of the file is encountered.
void reset()	Resets the input pointer to the previously set mark.
long skip(long <i>numBytes</i>)	Ignores (that is, skips) <i>numBytes</i> bytes of input, returning the number of bytes actually ignored.

OutputStream

- **OutputStream** is an **abstract class** that defines streaming byte output.
- Most of the methods in this class return **void** and throw an **IOException** in the case of errors.

methods in **OutputStream**.

```
void write(int b)
```

```
void write(byte buffer[ ])
```

```
void write(byte buffer[ ], int offset, int numBytes)
```

Writes a subrange of numBytes bytes from the array buffer, beginning at buffer[offset].

methods in **OutputStream**.

`void close( )`

`void flush( )`

Closes the output stream. Further write attempts will generate an **IOException**.

Finalizes the output state so that any buffers are cleared.

Table 19-2 shows the methods in **OutputStream**.

Method	Description
<code>void close()</code>	Closes the output stream. Further write attempts will generate an IOException .
<code>void flush()</code>	Finalizes the output state so that any buffers are cleared. That is, it flushes the output buffers.
<code>void write(int b)</code>	Writes a single byte to an output stream. Note that the parameter is an int , which allows you to call write() with expressions without having to cast them back to byte .
<code>void write(byte buffer[])</code>	Writes a complete array of bytes to an output stream.
<code>void write(byte buffer[], int offset, int numBytes)</code>	Writes a subrange of <i>numBytes</i> bytes from the array <i>buffer</i> , beginning at <i>buffer[offset]</i> .

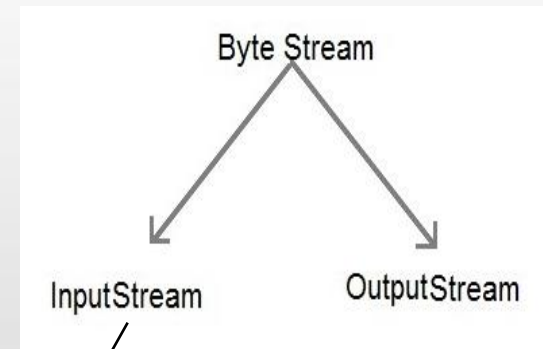
FileInputStream

- The **FileInputStream** class creates an **InputStream** that we can use to read bytes from a file.

`FileInputStream(String filepath)`

`FileInputStream(File fileObj)`

- Either can throw a **FileNotFoundException**.



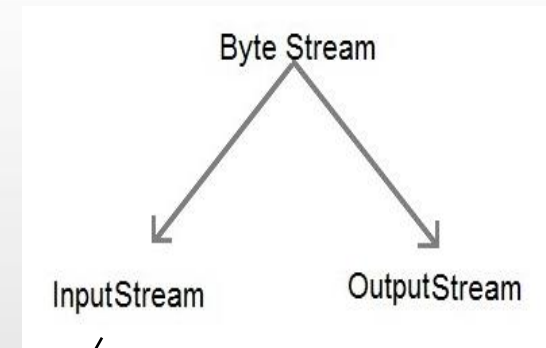
FileInputStream

Example

```
FileInputStream f0 = new FileInputStream ("c:/mit/ict/cce/prg1.txt");
```

```
File f = new File ("c:/mit/ict/cce/prg1.txt");
```

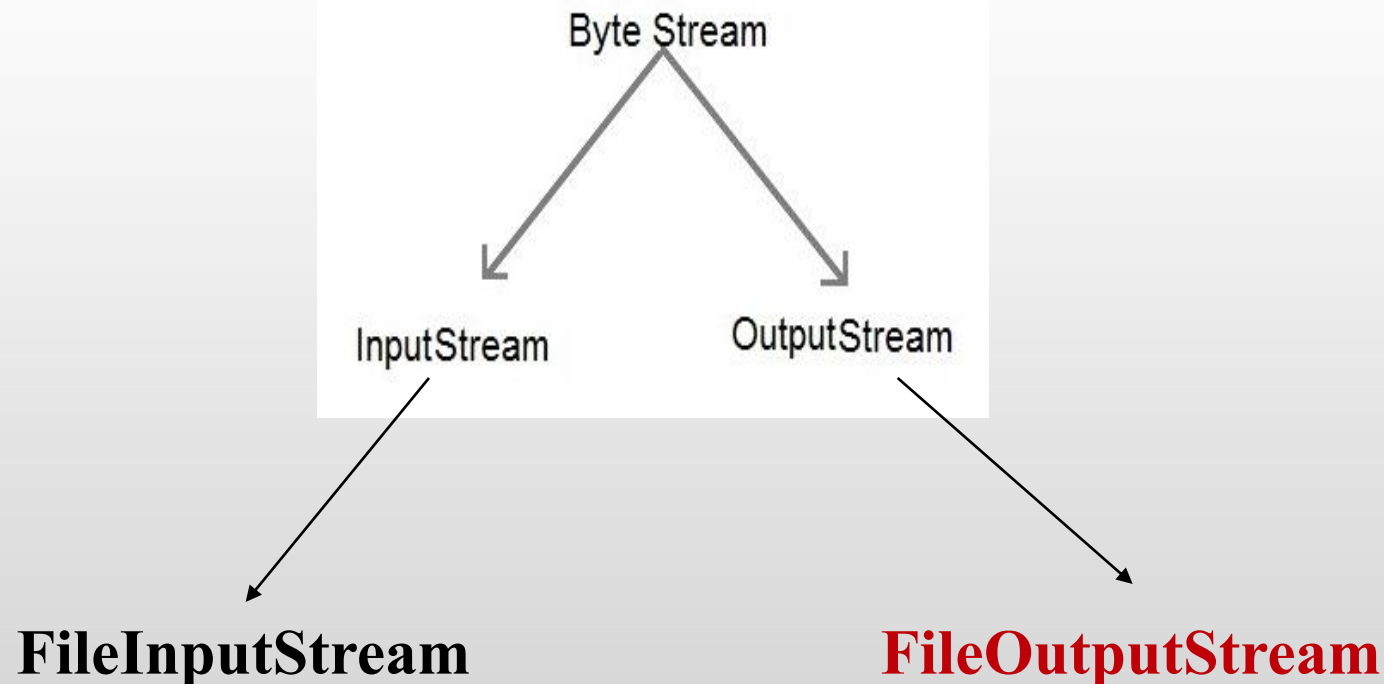
```
FileInputStream f1 = new FileInputStream(f);
```



FileInputStream

FileOutputStream

- **FileOutputStream** creates an **OutputStream** that you can use to write bytes to a file.



FileOutputStream

FileOutputStream(String *filePath*)

FileOutputStream(File *fileObj*)

FileOutputStream(String *filePath*, boolean *append*)

FileOutputStream(File *fileObj*, boolean *append*)

- They can throw a **FileNotFoundException**.
- If *append* is **true**, the file is opened in append mode.
- Creation of a **FileOutputStream** is not dependent on the file already existing.
- **FileOutputStream** will create the file before opening it for output when we create the object.
- In the case where you attempt to open a read-only file, an **IOException** will be thrown.

```
// Demonstrate FileOutputStream.
```

```
import java.io.*;
```

```
class FileOutputStreamDemo
```

```
{    public static void main(String args[]) throws IOException
```

```
{
```

```
    String source = "Now is the time for all good men\n"
```

```
        + " to come to the aid of their country\n"
```

```
        + " and pay their due taxes.";
```

```
    byte buf[] = source.getBytes();
```

```
    OutputStream f0 = new FileOutputStream("file1.txt");
```

```
    for (int i=0; i < buf.length; i += 2)
```

```
    {        f0.write(buf[i]);
```

```
    }
```

```
f0.close();
```

```
OutputStream f1 = new FileOutputStream("file2.txt");
```

```
f1.write(buf);
```

```
f1.close();
```

```
OutputStream f2 = new FileOutputStream("file3.txt");
```

```
f2.write(buf,buf.length-buf.length/4,buf.length/4);
```

```
f2.close();
```

```
}
```

```
}
```

- The first, **file1.txt**, will contain every other byte from the sample.
- The second, **file2.txt**, will contain the entire set of bytes.
- The third and last, **file3.txt**, will contain only the last quarter.

String source = "Now is the time for all good men\n"
+ " to come to the aid of their country\n"
+ " and pay their due taxes.";

```
Nwi h iefralgo e  
t oet h i ftercuty n a hi u ae.
```

Next, file2.txt:

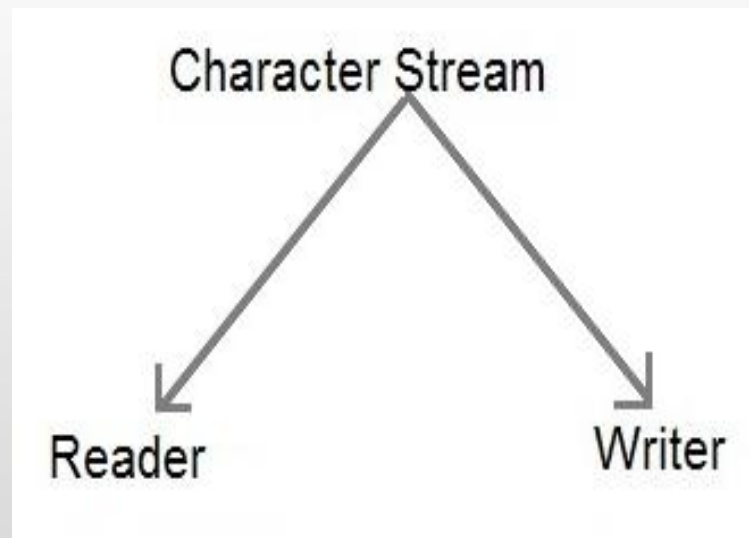
```
Now is the time for all good men  
to come to the aid of their country  
and pay their due taxes.
```

Finally, file3.txt:

```
nd pay their due taxes.
```


The Character Streams

- Top of the character stream hierarchies are the **Reader** and **Writer** abstract classes.



The Character Streams

- The byte stream classes provide functionality to handle any type of I/O operation, they cannot work directly with Unicode characters.
- Character streams : direct I/O support for characters.

NOTE :

- The character I/O classes were added by the 1.1 release of Java.
- Because of this, we may still find legacy code that uses byte streams where character streams would be more appropriate

Reader

- **Reader** is an abstract class that defines Java's model of streaming character input.

Reader methods

`int read( )`

`int read(char buffer[ ])`

`int read(char buffer[ ], int offset, int numChars)`

- Returns an integer representation of the next available character from the invoking input stream.
- Attempts to read up to `buffer.length` characters into `buffer` and returns the actual number of characters that were successfully read.
- Attempts to read up to `numChars` characters into `buffer` starting at `buffer[offset]`, returning the number of characters successfully read.
- `-1` is returned when the end of the file is encountered.

Reader methods

`long skip(long numBytes)`

`void close( )`

- Skips over numChars characters of input, returning the number of characters actually skipped.
- Closes the input source. Further read attempts will generate an IOException.

Reader methods

`boolean markSupported( )`

`void mark(int numChars)`

`void reset( )`

- Returns true if `mark( )`/`reset( )` are supported on this stream.
- Places a mark at the current point in the input stream that will remain valid until `numChars` characters are read.
- Resets the input pointer to the previously set mark.

Table 19-3 provides a synopsis of the methods in **Reader**.

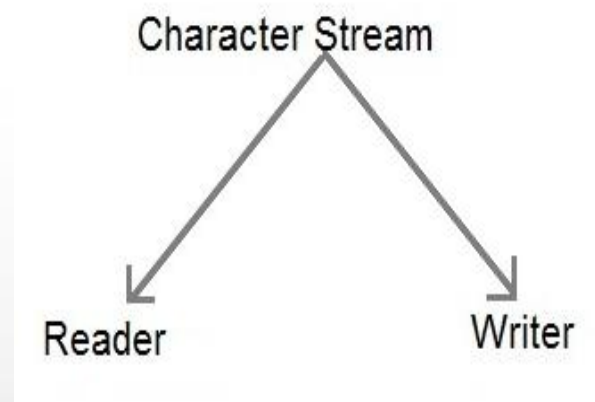
Method	Description
abstract void close()	Closes the input source. Further read attempts will generate an IOException .
void mark(int <i>numChars</i>)	Places a mark at the current point in the input stream that will remain valid until <i>numChars</i> characters are read.
boolean markSupported()	Returns true if mark() / reset() are supported on this stream.
int read()	Returns an integer representation of the next available character from the invoking input stream. -1 is returned when the end of the file is encountered.
int read(char <i>buffer</i> [])	Attempts to read up to <i>buffer.length</i> characters into <i>buffer</i> and returns the actual number of characters that were successfully read. -1 is returned when the end of the file is encountered.
abstract int read(char <i>buffer</i> [], int <i>offset</i> , int <i>numChars</i>)	Attempts to read up to <i>numChars</i> characters into <i>buffer</i> starting at <i>buffer[offset]</i> , returning the number of characters successfully read. -1 is returned when the end of the file is encountered.
boolean ready()	Returns true if the next input request will not wait. Otherwise, it returns false .
void reset()	Resets the input pointer to the previously set mark.
long skip(long <i>numChars</i>)	Skips over <i>numChars</i> characters of input, returning the number of characters actually skipped.

Writer

- **Writer** is an abstract class that defines streaming character output.
- All of the methods in this class throw an **IOException** in the case of errors.

Table 19-4 shows a synopsis of the methods in **Writer**.

Method	Description
Writer append(char <i>ch</i>)	Appends <i>ch</i> to the end of the invoking output stream. Returns a reference to the invoking stream.
Writer append(CharSequence <i>chars</i>)	Appends <i>chars</i> to the end of the invoking output stream. Returns a reference to the invoking stream.
Writer append(CharSequence <i>chars</i> , int <i>begin</i> , int <i>end</i>)	Appends the subrange of <i>chars</i> specified by <i>begin</i> and <i>end</i> -1 to the end of the invoking output stream. Returns a reference to the invoking stream.
abstract void close()	Closes the output stream. Further write attempts will generate an IOException .
abstract void flush()	Finalizes the output state so that any buffers are cleared. That is, it flushes the output buffers.
void write(int <i>ch</i>)	Writes a single character to the invoking output stream. Note that the parameter is an int , which allows you to call write with expressions without having to cast them back to char .



.....

FileReader

A dashed line representing the 'Reader' class, with a solid black arrow pointing down and to the left to the text 'FileReader'.

.....

FileWriter

A dashed line representing the 'Writer' class, with a solid black arrow pointing down and to the right to the text 'FileWriter'.

FileReader

- **FileReader** class creates a **Reader** that can be used to read the contents of a file.
- Its two most commonly used constructors are:

`FileReader(String filePath)`

`FileReader(File fileObj)`

- Either can throw a **FileNotFoundException**. Here, *filePath* is the full path name of a file, and *fileObj* is a **File** object that describes the file.

// Demonstrate FileReader.

```
import java.io.*;
```

```
class FileReaderDemo
```

```
{    public static void main(String args[]) throws IOException
```

```
{
```

```
    FileReader fr = new FileReader("FileReaderDemo.java");
```

```
    int i;
```

```
    while((i = fr.read()) != -1)
```

```
    {        System.out.println((char)i);
```

```
    }
```

```
    fr.close();
```

```
}}
```

FileWriter

- **FileWriter** creates a **Writer** that we can use to write to a file.
- Its most commonly used constructors are:

`FileWriter(String filePath)`

`FileWriter(String filePath, boolean append)`

`FileWriter(File fileObj)`

`FileWriter(File fileObj, boolean append)`

- They can throw an **IOException**.
- If *append* is **true**, then output is appended to the end of the file.
- Creation of a **FileWriter** is not dependent on the file already existing. **FileWriter** will create the file before opening it for output when you create the object. In the case where you attempt to open a read-only file, an **IOException** will be thrown.

// Demonstrate FileWriter.

```
import java.io.*;
```

```
class FileWriterDemo
```

```
{    public static void main(String args[]) throws IOException
```

```
{
```

```
    String source = "Now is the time for all good men\n"
```

```
    + " to come to the aid of their country\n"
```

```
    + " and pay their due taxes.";
```

```
    char buffer[] = new char[source.length()];
```

```
    source.getChars(0, source.length(), buffer, 0);
```

```
    FileWriter f0 = new FileWriter("file1.txt");
```

```
    for (int i=0; i < buffer.length; i += 2) {
```

```
        f0.write(buffer[i]);
```

```
    }
```

```
    f0.close();
```

```
FileWriter f1 = new FileWriter("file2.txt");  
f1.write(buffer);  
f1.close();
```

```
FileWriter f2 = new FileWriter("file3.txt");  
f2.write(buffer,buffer.length-buffer.length/4,buffer.length/4);  
f2.close();  
}  
}
```

The first, **file1.txt**, will contain every other character from the sample.
The second, **file2.txt**, will contain the entire set of characters.
Finally, the third, **file3.txt**, will contain only the last quarter

```
void write(int b);  
void write(byte buffer[]);  
void write(byte buffer[], int index, int size);
```


Differences

Aspect	Byte Stream	Character Stream
Unit of Data	Byte (8 bits)	Character (16 bits, Unicode)
Data Type	Binary data	Text data
Encoding	No character encoding	Uses character encoding (UTF-8, UTF-16)
Base Classes	<code>InputStream</code> , <code>OutputStream</code>	<code>Reader</code> , <code>Writer</code>
Example Classes	<code>FileInputStream</code> , <code>FileOutputStream</code>	<code>FileReader</code> , <code>FileWriter</code>
Suitable For	Images, audio, video, PDFs	Text files, documents, logs
Portability	Lower for text	Higher for text
Conversion	No encoding conversion	Handles encoding/decoding